Chapter 3 Normalization

Introduction

- Normalization is the process of organizing data in a database.
- This includes creating tables and establishing relationships between those tables according to rules designed both to protect the data and to make the database more flexible by eliminating redundancy and inconsistent dependency.
- What is an "inconsistent dependency"? While it is intuitive for a user to look in the Customers table for the address of a particular customer, it may not make sense to look there for the salary of the employee who calls on that customer.
- The employee's salary is related to, or dependent on, the employee and thus should be moved to the Employees table.
- Inconsistent dependencies can make data difficult to access because the path to find the data may be missing or broken.
- There are a few rules for database normalization. Each rule is called a "normal form."
- If the first rule is observed, the database is said to be in "first normal form."
- If the first three rules are observed, the database is considered to be in "third normal form."
- Although other levels of normalization are possible, third normal form is considered the highest level necessary for most applications.

Data Normalization

- A tool to validate and improve a logical design so that it satisfies certain constraints that avoid unnecessary duplication of data.
- The process of decomposing relations with anomalies to produce smaller, well structured.

Well Structured Relations

- A relation that contains minimal data redundancy and allows users to insert, delete, and update rows without causing data inconsistencies

Goal is to avoid anomalies

Insertion Anomaly	▪ adding new rows forces user to create duplicate data
Deletion Anomaly	▪ deleting rows may cause a loss of data that would be needed for other future rows
Modification /update Anomaly	▪ changing data in a row forces changes to other rows because of duplication

Example

emp_no	emp_name	emp_address	emp_dept
101	Ahmed	Zamalek	D01
101	Ahmed	Zamalek	D02
123	Sami	Dokki	D90
166	Yara	Helwan	D96
166	Yara	Helwan	D04

1. **Update anomaly:**
   - In the above table we have two rows for employee Ahmed as he belongs to two departments of the company.
   - If we want to update the address of Ahmed, then we must update the same in two rows or the data will become inconsistent.
   - If somehow, the correct address gets updated in one department but not in other then as per the database, Ahmed would be having two different addresses, which is not correct and would lead to inconsistent data.
2. **Insert anomaly:**
   - Suppose a new employee joins the company, who is under training and currently not assigned to any department then we would not be able to insert the data into the table if emp_dept field does not allow nulls.
3. **Delete anomaly:**
   - Suppose, if at a point of time the company closes the department D90 then deleting the rows that are having emp_dept as D90 would also delete the information of employee sami since he is assigned only to this department.
   - To overcome these anomalies, we need to normalize the data.

➢ **Why do these anomalies exist?**

- Because there are two themes (entity types) in this one relation.
- This results in data duplication and an unnecessary dependency between the entities.

Database Normal Forms

- Normal forms are used to eliminate or reduce redundancy in database tables.
- These are the most commonly used normal forms.
  - First normal form(1NF)
  - Second normal form(2NF)
  - Third normal form(3NF)
  - Boyce & Codd normal form (BCNF)
  - Fourth normal form(4NF)

First normal form(1NF)

- If a relation contains composite or multi-valued attribute, it violates first normal form, or a relation is in first normal form if it does not contain any composite or multi-valued attribute.
- A relation is in first normal form if every attribute in that relation is singled valued attribute.

For a table to be in the First Normal Form, it should follow the following 4 rules:

1. It should only have single (atomic) valued attributes.
   - Each column of your table should be single valued which means they should not contain multiple values.
2. Values stored in a column should be of the same domain.
   - This is more of a "Common Sense" rule. In each column the values stored must be of the same kind or type.
3. All the columns in a table should have unique names.
   - This rule expects that each column in a table should have a unique name.
   - This is to avoid confusion at the time of retrieving data or performing any other operation on the stored data.
   - If one or more columns have same name, then the DBMS system will be left confused.
4. And the order in which data is stored does not matter.

Employee	Age	Department
Melvin	32	Marketing, Sales
Edward	45	Quality Assurance
Alex	36	Human Resource

Not in 1NF Because of multivalued in department column

To make the table complies with 1NF we should have the data like

Employee	Age	Department
Melvin	32	Marketing
Melvin	32	Sales
Edward	45	Quality Assurance
Alex	36	Human Resource

➢ Second normal form(2NF)

- To be in second normal form, a relation must be in first normal form and relation must not contain any partial dependency.
- A relation is in 2NF if it has No Partial Dependency, i.e., no non-prime attribute (attributes which are not part of any candidate key) is dependent on any proper subset of any candidate key of the table.
- An attribute that is not part of any candidate key is known as non-prime attribute.

For a table to be in the Second Normal Form, it must satisfy two conditions:

1. The table should be in the First Normal Form.
2. There should be no Partial Dependency (remove Partial Dependency).

Example

teacher_id	subject	teacher_age
111	Maths	38
111	Physics	38
222	Biology	38
333	Physics	40
333	Chemistry	40

✓ The table is in 1 NF because each attribute has atomic values.

× However, it is not in 2NF because non-prime attribute teacher_age is dependent on teacher_id alone which is a proper subset of candidate key.

× This violates the rule for 2NF as the rule says, "no non-prime attribute is dependent on the proper subset of any candidate key of the table".

To make the table complies with 2NF we can break it in two tables like this:

teacher_details table:

teacher_id	teacher_age
111	38
222	38
333	40

teacher_subject table:

teacher_id	subject
111	Maths
111	Physics
222	Biology
333	Physics
333	Chemistry

➢ Third normal form(3NF)

- **A relation is in third normal form if there is no transitive dependency for non-prime attributes as well as it is in second normal form.**

A table is said to be in the Third Normal Form when,

1. **It is in the Second Normal form.**
2. **And, it does not have Transitive Dependency.**

Example:

- **Suppose a company wants to store the complete address of each employee, they create a table named employee_details that looks like this:**

emp_id	emp_name	emp_zip	emp_state	emp_city	emp_district
1001	John	282005	UP	Agra	Dayal Bagh
1002	Ajeet	222008	TN	Chennai	M-City
1006	Lora	282007	TN	Chennai	Urrapakkam
1101	Lilly	292008	UK	Pauri	Bhagwan
1201	Steve	222999	MP	Gwalior	Ratan

- × **Here, emp_state, emp_city & emp_district dependent on emp_zip.**
- × **And emp_zip is dependent on emp_id that makes non-prime attributes (emp_state, emp_city & emp_district) transitively dependent on super key (emp_id).**
- × **This violates the rule of 3NF.**

To make this table complies with 3NF we must break the table into two tables to remove the transitive dependency:

employee table:

emp_id	emp_name	emp_zip
1001	John	282005
1002	Ajeet	222008
1006	Lora	282007
1101	Lilly	292008
1201	Steve	222999

employee_zip table:

emp_zip	emp_state	emp_city	emp_district
282005	UP	Agra	Dayal Bagh
222008	TN	Chennai	M-City
282007	TN	Chennai	Urrapakkam
292008	UK	Pauri	Bhagwan
222999	MP	Gwalior	Ratan

- **By removing transitive dependency**
  1. **Amount of data duplication is reduced.**
  2. **Data integrity achieved.**

➢ Boyce & Codd normal form (BCNF)

- It is an advance version of 3NF that is why it is also referred as 3.5NF.
- BCNF is stricter than 3NF.
- A table complies with BCNF if it is in 3NF and for every functional dependency X→Y, X should be the super key of the table.

Example: Suppose there is a company where employees work in more than one department. They store the data like this:

emp_id	emp_nationality	emp_dept	dept_type	dept_no_of_emp
1001	Austrian	Production and planning	D001	200
1001	Austrian	stores	D001	250
1002	American	design and technical support	D134	100
1002	American	Purchasing department	D134	600

Functional dependencies in the table above:

- emp_id → emp_nationality
- emp_dept → {dept_type, dept_no_of_emp}
- Candidate key: {emp_id, emp_dept}
- The table is not in BCNF as neither emp_id nor emp_dept alone are keys.

To make the table comply with BCNF we can break the table in three tables like this:

emp_nationality table:

emp_id	emp_nationality
1001	Austrian
1002	American

emp_dept_mapping table:

emp_id	emp_dept
1001	Production and planning
1001	stores
1002	design and technical support
1002	Purchasing department

emp_dept table:

emp_dept	dept_type	dept_no_of_emp
Production and planning	D001	200
stores	**D001**	**250**
design and technical support	D134	100
Purchasing department	**D134**	**600**

Functional dependencies:

1. emp_id -> emp_nationality
2. emp_dept -> {dept_type, dept_no_of_emp}

Candidate keys:

- For first table: emp_id
- For second table: emp_dept
- For third table: {emp_id, emp_dept}

This is now in BCNF as in both the functional dependencies left side part is a key.

A.Hossam

➢ Fourth normal form (4NF)

- A table is said to be in the Fourth Normal Form when,
  1. It is in the Boyce-Codd Normal Form.
  2. And, it does not have Multi-Valued Dependency.
- A table is said to have multi-valued dependency, if the following conditions are true.
  1. For a dependency A → B, if for a single value of A, multiple value of B exists, then the table may have multi-valued dependency.
  2. Also, a table should have at-least 3 columns for it to have a multi-valued dependency.
  3. And, for a relation R(A,B,C), if there is a multi-valued dependency between, A and B, then B and C should be independent of each other.

If all these conditions are true for any relation(table), it is said to have multi-valued dependency.

Example:

- Below we have a college enrolment table with columns s_id, course, and hobby.

s_id	course	hobby
1	Science	Cricket
1	Maths	Hockey
2	C#	Cricket
2	Php	Hockey

- As you can see in the table above, student with s_id 1 has opted for two courses, Science and Maths, and has two hobbies, Cricket and Hockey.
- The two records for student with s_id 1, will give rise to two more records, as shown below, because for one student, two hobbies exist, hence along with both the courses, these hobbies should be specified.

s_id	course	hobby
1	Science	Cricket
1	Maths	Hockey
1	Science	Hockey
1	Maths	Cricket

- And, in the table above, there is no relationship between the columns course and hobby. They are independent of each other.
- So, there is multi-value dependency, which leads to un-necessary repetition of data and other anomalies as well.
- To make the above relation satisfy the 4th normal form, we can decompose the table into 2 tables.

CourseOpted Table

s_id	course
1	Science
1	Maths
2	C#
2	Php

And, Hobbies Table,

s_id	hobby
1	Cricket
1	Hockey
2	Cricket
2	Hockey

Now this relation satisfies the fourth normal form.

- **A table can also have functional dependency along with multi-valued dependency.**
- **In that case, the functionally dependent columns are moved in a separate table and the multi-valued dependent columns are moved to separate tables.**

The End of Chapter 3

لا تنسوا الدعاء لوالدي ولزميلنا عبدالله بالرحمة والمغفرة

A.Hossam